HÉRITAGE ET POLYMORPHISME

L’héritage

Relation entre classes

Permet à une classe de partager les attributs et les méthodes d’une autre classe

Favorise la réutilisation du code

Les constructeurs et le destructeur ne sont pas hérités.

Les méthodes `static` et `final` hérités ne peuvent être redéfinis

L’héritage

L’héritage est réalisé avec le mot clé **extends.**

La classe Object se trouve au sommet de la hiérarchie des classes.
Toutes les classes héritent de cette super-classe.

```
class Cercle extends Figure
{
…
}
```

Référence `super()`

Elle fait référence à la super classe.

Elle est utile pour appeler les constructeurs de la classe de base.

Elle doit occuper la première ligne du constructeur de la classe dérivée.

```
class Cercle extends
Figure {
…
Cercle() {
super();
rayon = 0.0;
}
```

```
class Figure {
…
Figure() {
centre = new
Point();
}
Figure (Point p) {
centre = p ;
}}
```

Référence `this`

Tout objet se voit lui-même il peut se renvoyer lui même en tant que résultat ou se passer en tant que paramètre d'un message en utilisant le pseudo-objet « `this` ».

```
class Point {
…
Point (int x, int y) {
this.x = x; this.y = y;
}
Point (Point p) {
this(p.x, p.y);
}
…
}
```

Relation entre constructeurs et méthodes

Si

- la première instruction du constructeur ne commence pas par le mot clé **super**

alors

- le constructeur par défaut de la classe mère est appelé et c'est une erreur s'il n'existe pas

Si

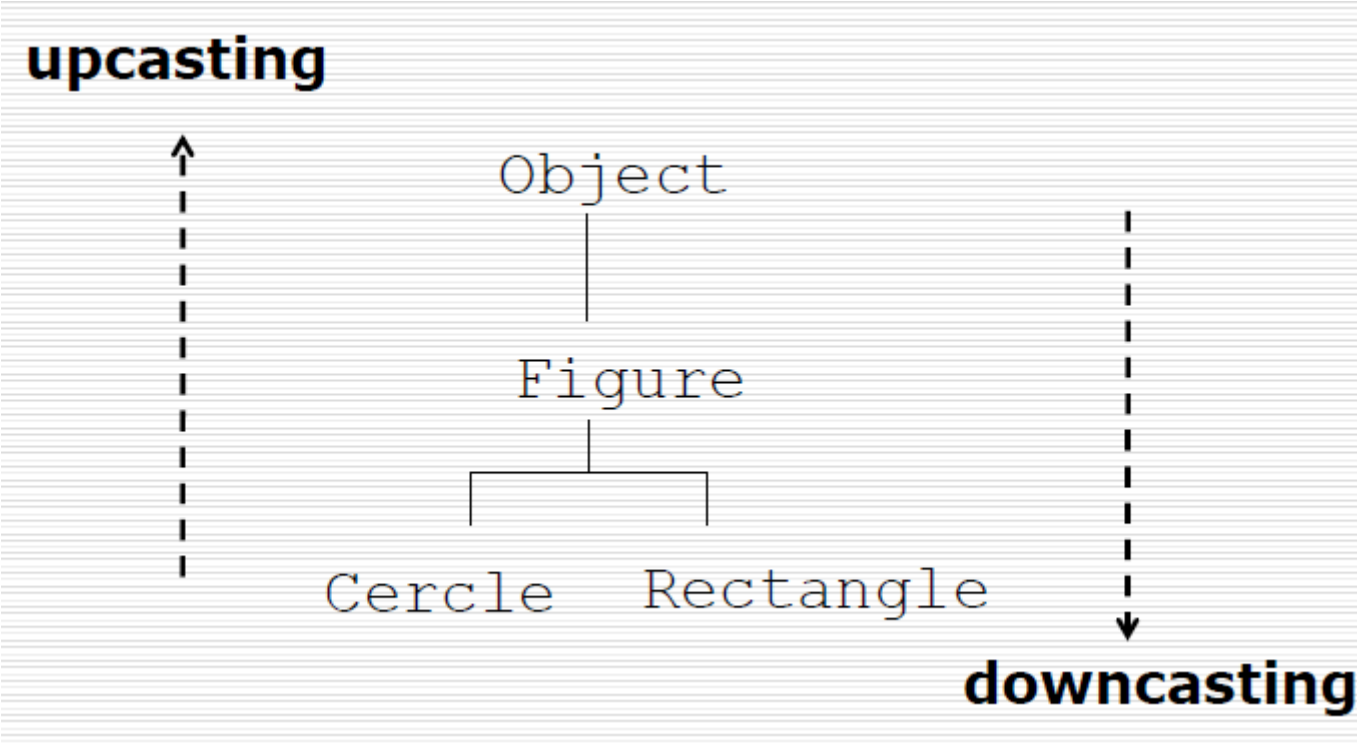
- deux méthodes ont la même signature dans une sous-classe et dans une super-classe

alors

- Java exécute la première trouvée en remontant la hiérarchie.

Conversion des objets

Un objet d'une classe peut être convertit en un objet d'une autre classe à condition que les deux classes soient liées par le mécanisme d'héritage.



L'opérateur `instanceof`

Il déterminé le type réel d'un objet.

Une référence objet peut être converti en un type plus spécifique si nécessiare.

```
Figure fg;
Cercle cerc = new Cercle();
Rectangle rec = new Rectangle();
fg = cerc;
if (fg instanceof Cercle) { …}
```

true

```
if (fg instanceof Figure) { …}
```

true

```
System.out.println(fg.getClass());
```

Cercle

Classe abstraite

Une classe abstraite ne peut pas être instanciée

Elle regroupe les informations communes des sous classes

Les méthodes abstraites doivent être implémentées par des sous classes.

Elle est déclarée avec le modificateur d'accès `abstract`

```
abstract class Figure {
 …
 abstract double surface();
}
```

Les interfaces

Une interface est semblable à une classe totalement abstraite

Toutes ses méthodes sont abstraites

Toutes les variables sont `public static final`

Une interface répertoire un ensemble de signatures de méthodes sans aucun code

Les interfaces

Les interfaces décrivent un aspect du comportement requis par différentes classes.

Toute classe qui implémente l'interface doit fournir les détails du code de toutes ses méthodes.

Une classe peut implémenter plusieurs interfaces (mai doit étendre qu'une seule classe)

```
class A extends B implements I1, I2, I3 {
…
}
```